

VR Racer Setup Anleitung

Stand: 28.04.2026

Diese Anleitung beschreibt, wie das VR_Racer Projekt auf einem Raspberry Pi 5 frisch eingerichtet, gestartet und mit Apple Vision Pro sowie einem PS5/DualSense Controller genutzt wird.

.env Settings

```
JWT_SECRET=super_secret_key

JWT_EXPIRE_MINUTES=5

ADMIN_G_PASS=Hallo123!

ADMIN_D_PASS=Pass123!


PORT=8443

HTTPS_ENABLED=1

HTTPS_CERT_FILE=certs/vrracer.crt

HTTPS_KEY_FILE=certs/vrracer.key

DB_PATH=data/users.db

KEY_FILE=data/secret.key


STREAM_BACKEND=mediamtx

MEDIAMTX_WEBRTC_URL=

CONTROLLER_DEVICE_PATH=

MOTOR_MAX_SPEED=0.65

SERVO_MAX_OUTPUT=0.25

SERVO_DETACH_ON_NEUTRAL=0

SERVO_UPDATE_EPSILON=0.005

STEERING_INVERTED=0

DEBUG_CONTROLLER=0
```

Mediamtx.yml Settings

```
#####  
# VR_Racer MediaMTX low-latency camera profile  
#  
# WebRTC player URL:  
#   https://<pi-ip>:8889/cam  
#  
# WHEP URL for a custom JS player:  
#   https://<pi-ip>:8889/cam/whep  
  
webrtc: true  
webrtcAddress: :8889  
webrtcEncryption: true  
webrtcServerKey: /etc/mediamtx/certs/vrracer.key  
webrtcServerCert: /etc/mediamtx/certs/vrracer.crt  
webrtcAllowOrigins: ['*']  
webrtcLocalUDPAddress: :8189  
webrtcLocalTCPAddress: ''  
webrtcIPsFromInterfaces: true  
webrtcAdditionalHosts: []  
  
hls: false  
rtmp: false  
srt: false  
  
paths:  
  cam:
```

```
source: rpiCamera
rpiCameraCamID: 0
rpiCameraWidth: 1280
rpiCameraHeight: 720
rpiCameraFPS: 30
rpiCameraCodec: auto
rpiCameraBitrate: 4000000
rpiCameraIDRPeriod: 30
rpiCameraHardwareH264Profile: baseline
rpiCameraSoftwareH264Profile: baseline
rpiCameraTextOverlayEnable: false
rpiCameraDenoise: "off"
```

1. Zielbild

- Der Raspberry Pi 5 hostet die Webseite lokal per HTTPS.
- MediaMTX liefert den Kamera-Stream per WebRTC unter `https://vrracer.local:8889/cam`.
- Die VR_Racer Webseite läuft unter `https://vrracer.local:8443`.
- Die Python-App ist für Login, UI und Controller-Start zuständig.
- Es wird genau eine Kamera genutzt.
- Cloudflare Tunnel wird nicht benötigt.

2. Voraussetzungen

- Raspberry Pi 5 mit Raspberry Pi OS.
- Kamera am Pi angeschlossen.
- SSH-Zugriff auf den Pi.
- Privates GitHub-Repo mit dem Projekt.
- Apple Vision Pro im gleichen lokalen Netzwerk wie der Pi.

- PS5/DualSense Controller.

3. Projekt auf dem Mac vorbereiten (optional)

Im Projektordner auf dem Mac:

```
cd /Users/georgzinn/PycharmProjects/VR_Racer  
git status
```

Änderungen committen und auf GitHub pushen:

```
git add .  
git commit -m "Update VR Racer setup"  
git push
```

Wichtig: Wenn Dateien wie `venv/`, `__pycache__`, `.DS_Store` oder lokale Datenbanken auftauchen, diese nicht mit hochladen.

```
git add app configs requirements scripts static templates  
Steuerung_VR_Racer server.py run_secure_cached.sh .env  
git commit -m "Update VR Racer"  
git push
```

4. Raspberry Pi komplett frisch einrichten

Per SSH auf den Pi verbinden:

```
ssh vrracersbs@vrracer.local
```

Falls `vrracer.local` nicht geht, die IP nutzen:

```
ssh vrracersbs@<pi-ip>
```

4.1 System aktualisieren und Pakete installieren

```
sudo apt update  
sudo apt upgrade -y  
sudo apt install -y git curl tar openssl python3-venv python3-pip  
python3-dev python3-picamera2 bluetooth bluez swig
```

4.2 Altes Projekt entfernen und neu clonen

Nur verwenden, wenn wirklich komplett neu installiert werden soll:

```
cd ~  
  
rm -rf VR_Racer  
  
git clone https://github.com/georg-Z14/VR_Racer.git  
  
cd ~/VR_Racer
```

Wenn GitHub nach einem Passwort fragt: GitHub akzeptiert kein normales Account-Passwort mehr. Dann einen GitHub Personal Access Token als Passwort verwenden.

4.3 Python venv erstellen

```
cd ~/VR_Racer  
  
python3 -m venv --system-site-packages venv  
  
source venv/bin/activate  
  
pip install --upgrade pip  
  
pip install -r requirements/pi.txt  
  
--system-site-packages ist wichtig, damit die venv die Raspberry-Pi-Systempakete wie picamera2 sehen kann.
```

4.4 Feste IP direkt auf dem Pi setzen (ohne DHCP-Reservierung im Router)

Wenn die IP nicht vom Router reserviert werden soll, kann die statische IP direkt auf dem Pi konfiguriert werden. Das ist genau dann sinnvoll, wenn der Pi immer dieselbe lokale IP haben soll, auch ohne DHCP-Reservierung im Router.

Erst die vorhandenen Verbindungen anzeigen:

```
nmcli con show
```

Beispiel: WLAN-Verbindung heißt `WLAN-RFYKV8` und soll auf `192.168.2.50` fest gesetzt werden.

```
sudo nmcli con mod "WLAN-RFYKV8" ipv4.addresses 192.168.2.50/24  
sudo nmcli con mod "WLAN-RFYKV8" ipv4.gateway 192.168.2.1  
sudo nmcli con mod "WLAN-RFYKV8" ipv4.dns "192.168.2.1 1.1.1.1"
```

```
sudo nmcli con mod "WLAN-RFYKV8" ipv4.method manual
```

Danach die Verbindung neu aufbauen:

```
sudo nmcli con down "WLAN-RFYKV8"
```

```
sudo nmcli con up "WLAN-RFYKV8"
```

Wichtig: Wenn du per SSH verbunden bist, bricht die Sitzung beim Umschalten des WLAN meist kurz ab. Das ist normal. Danach musst du dich mit der neuen festen IP wieder neu verbinden.

Neue Verbindung testen:

```
ssh vrracersbs@192.168.2.50
```

```
hostname -I
```

Wenn die Ausgabe `192.168.2.50` enthält, ist die feste IP aktiv.

Alternative: Falls lieber der grafische Raspberry-Pi-Weg genutzt wird, kann die statische IP auch über die Netzwerkeinstellungen auf dem Pi selbst gesetzt werden. Inhaltlich ist das dasselbe wie die `nmcli`-Befehle oben.

Wichtig für Zertifikate:

- Wenn du das Zertifikat nach dem Setzen der festen IP neu erzeugst, wird diese feste IP ebenfalls in das Zertifikat aufgenommen.
- Die robustere Variante bleibt trotzdem `https://vrracer.local:8443`, weil der Hostname unabhängig von der IP ist.
- Wenn ihr die feste IP direkt im Browser nutzen wollt, sollte nach der IP-Änderung das Zertifikat einmal neu erzeugt werden.

5. .env konfigurieren

Die Datei liegt im Projektroot:

```
cd ~/VR_Racer
```

```
nano .env
```

Empfohlener Inhalt:

```
JWT_SECRET=super_secret_key
```

```
JWT_EXPIRE_MINUTES=5
```

ADMIN_G_PASS=Hallo123!

ADMIN_D_PASS=Pass123!

PORT=8443

HTTPS_ENABLED=1

HTTPS_CERT_FILE=certs/vrracer.crt

HTTPS_KEY_FILE=certs/vrracer.key

DB_PATH=data/users.db

KEY_FILE=data/secret.key

STREAM_BACKEND=mediamtx

MEDIAMTX_WEBRTC_URL=

CONTROLLER_DEVICE_PATH=

MOTOR_MAX_SPEED=0.65

SERVO_MAX_OUTPUT=0.25

SERVO_DETACH_ON_NEUTRAL=0

SERVO_UPDATE_EPSILON=0.005

STEERING_INVERTED=0

DEBUG_CONTROLLER=0

5.1 Bedeutung der wichtigen .env Werte

- PORT=8443: HTTPS-Port der Webseite.
- HTTPS_ENABLED=1: Webseite startet mit HTTPS.
- STREAM_BACKEND=mediamtx: Kamera kommt über MediaMTX.
- MEDIAMTX_WEBRTC_URL=: leer lassen. Dann nutzt der Browser automatisch denselben Host wie die Webseite.
- CONTROLLER_DEVICE_PATH=: leer lassen. Der Code sucht automatisch das richtige DualSense-Gamepad. Nicht fest auf /dev/input/event13 setzen.

- `MOTOR_MAX_SPEED=0.65`: begrenzt die Maximalgeschwindigkeit auf 65 Prozent.
- `SERVO_MAX_OUTPUT=0.25`: begrenzt den Servo-Lenkausschlag auf 25 Prozent pro Richtung.
- `SERVO_DETACH_ON_NEUTRAL=0`: Servo bleibt in Neutralstellung aktiv auf Mitte, damit die Lenkung nicht nach einiger Zeit aussetzt.
- `SERVO_UPDATE_EPSILON=0.005`: kleinere Lenkeingaben werden sauberer erkannt.
- `STEERING_INVERTED=0`: falls links/rechts vertauscht ist, auf 1 setzen.
- `DEBUG_CONTROLLER=0`: auf 1 setzen, wenn Rohwerte des Controllers geloggt werden sollen.

Nach Änderungen an `.env`:

```
sudo systemctl restart vr-racer-server.service
sudo systemctl restart vr-racer-controller.service
```

6. HTTPS Zertifikat erstellen

```
cd ~/VR_Racer
chmod +x scripts/*.sh
./scripts/generate_local_https_cert.sh
```

Erzeugte Dateien:

```
certs/vrracer.crt
certs/vrracer.key
```

6.1 Zertifikat auf den Mac kopieren

Wichtig: `scp` muss auf dem Mac ausgeführt werden, nicht innerhalb der SSH-Sitzung auf dem Pi.

```
scp vrracersbs@vrracer.local:~/VR_Racer/certs/vrracer.crt
~/Downloads/vrracer.crt
```

Falls `vrracer.local` nicht geht:


```
scp vrracersbs@<pi-ip>:~/VR_Racer/certs/vrracer.crt  
~/Downloads/vrracer.crt
```

6.2 Zertifikat auf der Vision Pro vertrauen

- 1 `vrracer.crt` vom Mac auf die Vision Pro bringen, z. B. per iCloud Drive, AirDrop oder Dateien-App.
- 2 Auf der Vision Pro die Datei `vrracer.crt` öffnen und Profil installieren.
- 3 In den Einstellungen das installierte Zertifikat als vertrauenswürdig aktivieren.
- 4 Danach Safari komplett schließen und neu öffnen.

Wenn sich die IP des Pi ändert, funktioniert `vrracer.local` weiter, solange mDNS im Netzwerk funktioniert. Falls ihr direkt die IP nutzt und die IP ändert sich, muss das Zertifikat neu erzeugt und neu auf der Vision Pro vertraut werden.

7. MediaMTX installieren

```
cd ~/VR_Racer  
./scripts/install_mediamtx.sh  
sudo systemctl start mediamtx.service
```

Status prüfen:

```
systemctl status mediamtx.service
```

Logs live anzeigen:

```
journalctl -u mediamtx.service -f
```

Direkter Kamera-Test:

```
https://vrracer.local:8889/cam
```

7.1 Kamera-Auflösung ändern

Die Kamera wird in dieser Datei konfiguriert:

```
configs/mediamtx.yml
```

Aktuelles Profil:

```
rpiCameraCamID: 0
```

```
rpiCameraWidth: 1280
rpiCameraHeight: 720
rpiCameraFPS: 30
rpiCameraBitrate: 4000000
```

Nach Änderungen an configs/mediamtx.yml:

```
cd ~/VR_Racer
sudo cp configs/mediamtx.yml /etc/mediamtx/mediamtx.yml
sudo systemctl restart mediamtx.service
```

8. Autostart installieren

Der Autostart legt drei Services an:

- mediamtx.service: Kamera-Stream.
- vr-racer-server.service: HTTPS-Webseite.
- vr-racer-controller.service: PS5-Controller-Steuerung.

```
cd ~/VR_Racer
chmod +x scripts/*.sh
./scripts/install_autostart.sh
```

Direkt starten:

```
sudo systemctl start mediamtx.service
sudo systemctl start vr-racer-server.service
sudo systemctl start vr-racer-controller.service
```

Nach einem Reboot starten die Services automatisch:

```
sudo reboot
```

9. Webseite starten und testen

Status prüfen:

```
systemctl status mediamtx.service
systemctl status vr-racer-server.service
```

```
systemctl status vr-racer-controller.service
```

Webseite aufrufen:

```
https://vrracer.local:8443
```

Direkten MediaMTX Stream testen:

```
https://vrracer.local:8889/cam
```

Wenn `vrracer.local` nicht geht, die IP nutzen:

```
hostname -I
```

```
https://<pi-ip>:8443
```

```
https://<pi-ip>:8889/cam
```

10. Vision Pro nutzen

- 1 Vision Pro und Pi müssen im gleichen Netzwerk sein.
- 2 Zertifikat `vrracer.crt` muss auf der Vision Pro vertraut sein.
- 3 Safari auf der Vision Pro öffnen.
- 4 `https://vrracer.local:8443` öffnen.
- 5 Einloggen.
- 6 Stream starten.
- 7 VR/WebXR-Modus aktivieren.

Aktuelle WebXR-Schärfe-Defaults im Code:

```
xrFramebufferScale=1.6
```

```
xrCinemaScale=1.0
```

```
xrDistance=3.2
```

11. Neuen PS5 Controller koppeln

11.1 Wenn der Controller vorher mit dem Mac gekoppelt war

Auf dem Mac den Controller aus Bluetooth entfernen:

- 1 macOS Systemeinstellungen öffnen.
- 2 Bluetooth öffnen.

3 DualSense Wireless Controller **suchen**.

4 Gerät entfernen oder ignorieren.

macOS hat keinen verlässlichen Standard-Bash-Befehl zum Entkoppeln von Bluetooth-Geräten. Deshalb ist der GUI-Weg auf dem Mac am sichersten.

11.2 Controller auf dem Pi koppeln

Controller in Pairing-Modus bringen: PS-Taste + Create/Share-Taste halten, bis die LED schnell blinkt.

Auf dem Pi:

```
bluetoothctl
```

In bluetoothctl:

```
power on
```

```
agent on
```

```
default-agent
```

```
scan on
```

Wenn der Controller angezeigt wird, die MAC-Adresse merken, z. B.

AA:BB:CC:DD:EE:FF. Dann:

```
pair AA:BB:CC:DD:EE:FF
```

```
trust AA:BB:CC:DD:EE:FF
```

```
connect AA:BB:CC:DD:EE:FF
```

```
scan off
```

```
quit
```

Controller-Service neu starten:

```
sudo systemctl restart vr-racer-controller.service
```

```
journalctl -u vr-racer-controller.service -f
```

Im Log sollte stehen:

```
Verbunden mit: DualSense Wireless Controller
```

```
Lenkung ABS_X: min=0 center=127.5 max=255
```

```
L2: min=0 neutral=0 max=255 richtung=normal
```

```
R2: min=0 neutral=0 max=255 richtung=normal
```

11.3 Controller-Rechte setzen

Falls keine Controller- oder GPIO-Rechte vorhanden sind:

```
sudo usermod -aG input,gpio vrracersbs
```

Danach neu einloggen oder rebooten:

```
sudo reboot
```

11.4 Controller per Tastenkombination abmelden

Motor und Servo stoppen und Controller-Steuerung beenden:

PS + Options

Falls die PS-Taste nicht erkannt wird:

Create/Share + Options

Controller-Service wieder starten:

```
sudo systemctl start vr-racer-controller.service
```

12. Updates vom GitHub Repo auf dem Pi holen

Normaler Update-Ablauf:

```
cd ~/VR_Racer
```

```
git pull
```

```
sudo systemctl restart vr-racer-server.service
```

```
sudo systemctl restart vr-racer-controller.service
```

Wenn `.env` lokale Änderungen blockiert und die GitHub-Version genutzt werden soll:

```
cd ~/VR_Racer
```

```
git checkout -- .env
```

```
git pull
```

```
sudo systemctl restart vr-racer-server.service
```

```
sudo systemctl restart vr-racer-controller.service
```

Wenn configs/mediamtx.yml aktualisiert wurde:

```
cd ~/VR_Racer
```

```
git pull
```

```
sudo cp configs/mediamtx.yml /etc/mediamtx/mediamtx.yml
```

```
sudo systemctl restart mediamtx.service
```

13. Fehlerdiagnose

13.1 Webseite geht nicht

```
systemctl status vr-racer-server.service
```

```
journalctl -u vr-racer-server.service -f
```

13.2 Stream geht nicht

```
systemctl status mediamtx.service
```

```
journalctl -u mediamtx.service -f
```

Direkt testen:

```
https://vrracer.local:8889/cam
```

13.3 Controller geht nicht

```
systemctl status vr-racer-controller.service
```

```
journalctl -u vr-racer-controller.service -f
```

Input-Geräte anzeigen:

```
ls -l /dev/input/
```

```
cat /proc/bus/input/devices
```

Wenn Motion-Sensor statt Gamepad genutzt wird, sicherstellen:

```
CONTROLLER_DEVICE_PATH=
```

13.4 Auto lenkt falsch herum

In .env setzen:

```
STEERING_INVERTED=1
```

Danach:

```
sudo systemctl restart vr-racer-controller.service
```

13.5 Auto ist zu schnell

In `.env` reduzieren:

```
MOTOR_MAX_SPEED=0.45
```

Danach:

```
sudo systemctl restart vr-racer-controller.service
```

14. Schnellstart nach erfolgreicher Installation

```
cd ~/VR_Racer
```

```
sudo systemctl restart mediamtx.service
```

```
sudo systemctl restart vr-racer-server.service
```

```
sudo systemctl restart vr-racer-controller.service
```

Dann auf der Vision Pro:

```
https://vrracer.local:8443
```